Lezione 4 – Casting, wrapper, `Arrays`, modularità

Conversioni di tipo, classi wrapper, la classe `Arrays` e la programmazione modulare

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

6 ottobre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 3** (*15 min*)
   - uno di voi alla lavagna con `Statistiche`, `Istogramma` o `Simmetrica`: lo commentiamo insieme
2. **Tipi: aspetti avanzati** (*20 min*)
   - conversioni implicite e cast esplicito; `char` come numero (Unicode)
   - classi wrapper, autoboxing e auto-unboxing; la classe `Character`
3. `Arrays` **e varargs** (*10 min*)
   - stampare, copiare, confrontare, ordinare, cercare; argomenti a lunghezza variabile
4. **Programmazione modulare** (*15 min*)
   - astrazione, modulo (interfaccia e corpo), information hiding, coesione e accoppiamento
5. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Statistiche`, un modulo con `Arrays` e varargs
   - *Insieme, passo passo*: `Frequenze` – da un `main` che fa tutto a moduli coesi
6. **Wrap-up e Homework 4** (*10 min*)

Conversioni implicite (*widening*) – `Widening.java`

```java
public static double meta(double x) {
    return x / 2;
}

public static void main(String[] args) {
    char c = 'A';
    int i = c;          // char -> int: 65
    double d = i;       // int -> double: 65.0
    System.out.println(i + " " + d);

    System.out.println(c + 1);      // 66
    System.out.println(i + 0.5);    // 65.5
    System.out.println(7 / 2);      // 3
    System.out.println(7 / 2.0);    // 3.5
    System.out.println(meta(7));    // 3.5

    // int n = d;     // ERRORE: double -> int
    // char k = i;    // ERRORE: int -> char
}
```



Verso un tipo che contiene *tutti* i valori di quello di partenza la conversione è automatica:

- nell'**assegnazione**: `int i = c`, `double d = i`
- nelle **espressioni**: con operandi di tipo diverso vince il tipo più **capiente**. `char + int` è un `int`; `int + double` è un `double`
- nella **chiamata**: `meta(7)` riceve `7.0`

Nel verso opposto il compilatore si rifiuta. `boolean` non si converte in nessun altro tipo, in nessun modo.

Il cast esplicito (*narrowing*) – `Narrowing.java`

```
double prezzo = 9.99;
int euro = (int) prezzo;              // 9: tronca
int vicino = (int) (prezzo + 0.5);    // 10
System.out.println(euro + " " + vicino);
System.out.println((int) -9.99);      // -9

int somma = 59;
int n = 6;
System.out.println(somma / n);                // 9
System.out.println((double) somma / n);       // 9.83...
System.out.println((double) (somma / n));     // 9.0

System.out.println((int) 3e10);       // 2147483647
char strano = (char) 2000000000;
System.out.println((int) strano);     // 37888
```

`(tipo) espressione`: la conversione la chiedo io, e la perdita di informazione è una mia responsabilità.

Da `double` a `int` la parte decimale viene **troncata** (verso lo zero), non arrotondata. Per l'intero più vicino a un numero positivo: `(int) (x + 0.5)`.

Il cast agisce su ciò che lo segue. `(double) somma / n` converte `somma` e poi divide: divisione reale. `(double) (somma / n)` divide fra interi e converte il risultato: troppo tardi.

Fuori dai limiti nessun errore, nessun avviso:

- `double` → `int`: si ottiene il limite, `Integer.MAX_VALUE`
- `int` → `char`: restano i 16 bit meno significativi

char è un numero: il codice Unicode – Caratteri.java

```
char c = 'a';
System.out.println((int) c);           // 97
System.out.println((char) 98);         // b
System.out.println(c + 1);             // 98: int
System.out.println((char) (c + 1));    // b

// c = c + 1;    // ERRORE: int -> char
c++;             // ok: ora c vale 'b'
c += 2;          // ok: ora c vale 'd'
System.out.println(c);                 // d

char accentata = '\u00E8';   // la e accentata
System.out.println((int) accentata);  // 232

System.out.println('7' - '0');         // 7
System.out.println('e' - 'a');         // 4
System.out.println((char) ('a' + 4)); // e
System.out.println((char) ('e' - 'a' + 'A')); // E
```

Un `char` è un intero senza segno a 16 bit: il **codice Unicode** del carattere. `'a'` e 97 sono lo stesso valore.

`'0'...'9'`	`'A'...'Z'`	`'a'...'z'`
48...57	65...90	97...122

- cifre e lettere hanno codici **consecutivi**: `c - '0'` è il valore di una cifra, `c - 'a'` la posizione di una lettera, `(char) ('a' + k)` la lettera di posizione `k`
- `c + 1` è un `int`: per riavere un `char` serve il cast. `c++` e `c += 2` lo contengono già
- `'\uXXXX'`: un carattere scritto con il suo codice, in esadecimale

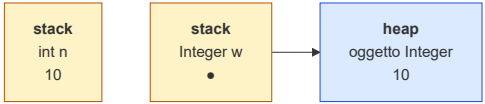
Le classi wrapper – `Wrapper.java`

```
int n = 10;                          // il valore
Integer w = Integer.valueOf(10);  // un oggetto

// metodi di istanza: w e' un oggetto
System.out.println(w.intValue() + n);        // 20
System.out.println(w.doubleValue() / 4);     // 2.5
System.out.println(w.toString().length());  // 2

// costanti e metodi static: il corredo del tipo
int max = Integer.MAX_VALUE;    // 2147483647
int k = Integer.parseInt("42");            // 42
double x = Double.parseDouble("2.5");      // 2.5
String t = Integer.toString(42);           // "42"
String bin = Integer.toBinaryString(10);  // "1010"
System.out.println(max + " " + k + " " + x);
System.out.println(t + 1 + " " + bin);  // 421 1010
```

Sono **immutabili**, come `String`: nessun metodo cambia il valore contenuto.



Ogni tipo primitivo ha la sua classe *wrapper* (in `java.lang`): `Boolean`, `Character`, `Integer`, `Double`, e così per gli altri quattro.

A cosa servono

- a usare un valore primitivo dove sono ammessi solo **oggetti** (le strutture dati delle lezioni 11 e 12)
- i primitivi non hanno metodi: costanti e metodi `static` che li riguardano stanno nel wrapper

Autoboxing e auto-unboxing – `Boxing.java`

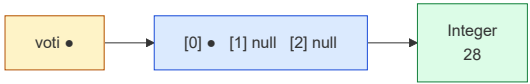
```
public static void azzera(Integer x) {
    x = 0;     // cambia solo la copia
}

public static void main(String[] args) {
    Integer w = 10;      // boxing
    int n = w;           // unboxing
    Integer s = w + n;   // unboxing e boxing
    System.out.println(s);              // 20

    azzera(w);
    System.out.println(w);              // 10

    Integer a = 1000;
    Integer b = 1000;
    System.out.println(a == b);         // false
    System.out.println(a.equals(b));    // true

    Integer[] voti = new Integer[3];  // 3 null!
    voti[0] = 28;                     // boxing
    int primo = voti[0];              // unboxing
    System.out.println(primo);        // 28
    // int x = voti[1];   // NullPointerException
}
```



Il compilatore converte da solo fra primitivo e wrapper: **boxing** (`Integer.valueOf(10)`) e **unboxing** (`w.intValue()`).

- restano **oggetti**: == confronta i riferimenti, il valore si confronta con `equals`. Fra -128 e 127 == fra `Integer` dà `true` (una cache, come il pool delle `String`); fra `Double` mai. Fra primitivi == resta corretto
- un `Integer[]` nasce pieno di `null`, non di zeri (un `int[]` sì): l'unboxing di `null` ferma il programma
- `azzera` non cambia `w`: riceve una copia del riferimento, e `x = 0` la fa puntare a un altro oggetto

La classe Character – InfoCarattere.java

```
String s = "Aula T2, ore 9:30";
int lettere = 0;
int cifre = 0;
int maiuscole = 0;
for (int i = 0; i < s.length(); i++) {
    char c = s.charAt(i);
    if (Character.isLetter(c)) {
        lettere++;
    }
    if (Character.isDigit(c)) {
        cifre++;
    }
    if (Character.isUpperCase(c)) {
        maiuscole++;
    }
}
System.out.println(lettere + " lettere, " + cifre
        + " cifre, " + maiuscole + " maiuscole");

char m = Character.toUpperCase('a');        // 'A'
char r = Character.toUpperCase('7');        // '7'
System.out.println(m + " " + r);
```

```
8 lettere, 4 cifre, 2 maiuscole
A 7
```

Il wrapper di `char`. Quasi tutti i suoi metodi sono `static` e ricevono il carattere come argomento:

lettera? cifra?	`isLetter`, `isDigit`
lettera o cifra?	`isLetterOrDigit`
maiuscola? minuscola?	`isUpperCase`, `isLowerCase`
spazio, tabulazione, a capo?	`isWhitespace`
conversione	`toUpperCase`, `toLowerCase`

- `toUpperCase` restituisce l'argomento stesso se non esiste una maiuscola corrispondente
- `isLetter` conosce *tutte* le lettere Unicode, accentate comprese; `c >= 'a' && c <= 'z'` solo quelle 26

java.util.Arrays: stampare, copiare, confrontare – UsaArrays.java

```java
import java.util.Arrays;

int[] a = {4, 5, 1, 13, 0};
System.out.println(a);              // riferimento!
System.out.println(Arrays.toString(a));

int[] b = Arrays.copyOf(a, a.length);  // copia vera
b[0] = 99;
System.out.println(a[0]);           // a non cambia
int[] lungo = Arrays.copyOf(a, 7);
System.out.println(Arrays.toString(lungo));

int[] c = {4, 5, 1, 13, 0};
System.out.println(a == c);
System.out.println(a.equals(c));  // come ==
System.out.println(Arrays.equals(a, c));

int[] d = new int[4];
Arrays.fill(d, 7);
System.out.println(Arrays.toString(d));
```

`Arrays` **non è un wrapper**: è una classe di soli metodi `static`, che lavorano sugli array ricevuti come argomento.

```
[I@659e0bfd
[4, 5, 1, 13, 0]
4
[4, 5, 1, 13, 0, 0, 0]
false
false
true
[7, 7, 7, 7]
```

- `toString`: il contenuto come stringa
- `copyOf(a, n)`: un array *nuovo* di `n` elementi (zeri in coda se `n` è maggiore)
- `Arrays.equals(a, c)`: confronta il contenuto
- `fill`: lo stesso valore in ogni elemento

Perché `a.equals(c)` **dà** `false`**?** `equals` confronta il contenuto solo nelle classi che lo definiscono così, come `String` e i wrapper. Gli array non lo fanno: il loro `equals` si comporta come `==`. Per questo esiste `Arrays.equals`.

Arrays: ordinare e cercare – OrdinaCerca.java

```java
int[] v = {12, 7, 3, 21, 9};
Arrays.sort(v);          // MODIFICA v
System.out.println(Arrays.toString(v));
System.out.println(Arrays.binarySearch(v, 12));
System.out.println(Arrays.binarySearch(v, 8));

String[] nomi = {"Marta", "anna", "Luca"};
Arrays.sort(nomi);       // usa compareTo
System.out.println(Arrays.toString(nomi));

int[][] m = {{1, 2}, {3, 4}};
System.out.println(Arrays.toString(m));
System.out.println(Arrays.deepToString(m));
```

```
[3, 7, 9, 12, 21]
3
-3
[Luca, Marta, anna]
[[I@15db9742, [I@6d06d69c]
[[1, 2], [3, 4]]
```

- `sort` ordina in senso crescente, **sul posto**: è `void` e modifica l'array del chiamante, attraverso la copia del riferimento (come `inverti` nella lezione 3). Con `String` e i wrapper, immutabili, nessun metodo può farlo
- per conservare l'ordine originale: `copyOf`, poi `sort` sulla copia
- `binarySearch` vuole un array **ordinato**: come cercare una parola nel dizionario, si apre a metà e si prosegue nella metà giusta. Restituisce l'indice, o un numero negativo se l'elemento non c'è (lezione 11)
- per le matrici: `deepToString` e `deepEquals`

leggono	`toString`, `equals`, `binarySearch`
modificano	`sort`, `fill`
creano	`copyOf`

Argomenti a lunghezza variabile (*varargs*) – `Varargs.java`

```java
/** Pre: almeno un numero. */
public static double media(double... numeri) {
    double somma = 0;
    for (double x : numeri) {   // un double[]
        somma += x;
    }
    return somma / numeri.length;
}

/** Il primo argomento (a) e' obbligatorio. */
public static int massimo(int a, int... altri) {
    int max = a;
    for (int x : altri) {
        if (x > max) {
            max = x;
        }
    }
    return max;
}
```

`tipo... nome`: il metodo accetta **zero o più** argomenti di quel tipo. Dentro il metodo `nome` è un normale array.

```java
double[] voti = {28, 30, 24};
double a = media(10, 40);           // 25.0
double b = media(10, 40, 50, 75);   // 43.75
double c = media(voti);             // 27.33
double d = media();                 // NaN
int m = massimo(3, 9, 4);           // 9
// massimo();     // ERRORE: manca il primo
System.out.println(a + " " + b + " " + c);
System.out.println(d + " " + m);
```

- variabile è il **numero di argomenti**, non l'array: a ogni chiamata il compilatore ne crea uno della lunghezza giusta, che poi, come ogni array, non cambia più. Se ne può anche passare uno già pronto
- un solo parametro varargs, e **per ultimo**
- `media()` compila, e `0.0 / 0` dà `NaN`: un parametro normale davanti, come in `massimo`, rende obbligatorio il primo argomento
- `printf` è un metodo varargs

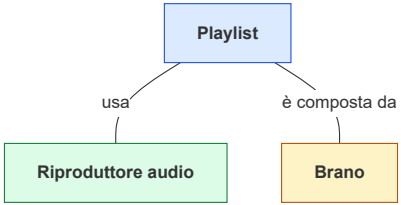
Programmazione modulare: suddividere un sistema in parti

Un sistema software complesso non si scrive, né si capisce, tutto insieme: lo si **suddivide** in parti più piccole, i *moduli*, ciascuna con un compito ben definito, e si stabilisce come queste parti sono collegate fra loro. L'insieme delle parti e dei loro collegamenti è l'*architettura* del sistema.

Ne usate dalla prima lezione: `Scanner`, `Math`, `String` sono moduli scritti da altri. Sapete cosa fanno e come si chiamano i loro metodi; il loro codice non l'avete mai letto.

Che cosa si guadagna

- **riuso**: lo stesso modulo serve a più programmi
- **modificabilità**: si corregge o si migliora un modulo senza toccare gli altri
- **leggibilità**: si capisce, e si prova, un pezzo alla volta
- **sviluppo separato**: più persone lavorano insieme, ciascuna sul suo modulo



Un lettore musicale suddiviso in tre moduli, e i due modi in cui due moduli possono essere collegati:

- **usa**: la playlist si serve del riproduttore audio per suonare
- **è composta da**: la playlist è fatta di brani

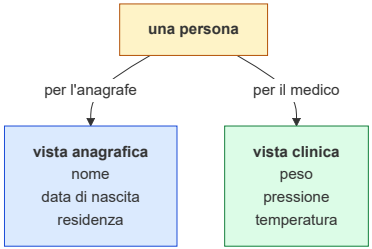
Programmazione modulare: astrarre

Astrazione

Considerare le proprietà *rilevanti* di un'entità, ignorando i dettagli non essenziali.

La stessa persona ha una vista *anagrafica* (nome, data di nascita, residenza) e una vista *clinica* (peso, pressione, temperatura). Nessuna delle due è "tutta" la persona: ciascuna tiene ciò che serve a chi la usa.

- **astrazione sul controllo**: una funzionalità, separata dai dettagli di come è realizzata. Sono i metodi: di `Arrays.sort` sapete *cosa* fa, non *come*
- **astrazione sui dati**: un dato, separato dai dettagli di come è memorizzato. Di una `String` sapete *cosa* rappresenta (una sequenza di caratteri) e quali operazioni permette (`length`, `charAt`...), non *come* i caratteri sono tenuti in memoria. Sono i tipi: i nostri li definiremo con le classi



L'astrazione fondamentale dei linguaggi procedurali, come il C, è quella sul controllo; quella dei linguaggi a oggetti, come Java, è l'astrazione sui dati.

Programmazione modulare: il modulo

Modulo

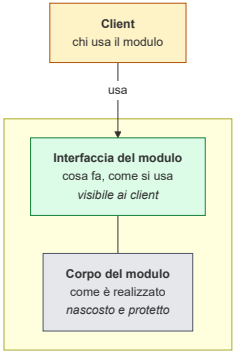
Una delle parti in cui il sistema è suddiviso: realizza un'astrazione. Chi lo usa, un altro modulo o un programma, è un suo **client**.

L'astrazione può essere solo sul controllo (`Math`: operazioni) oppure sui dati e sul controllo insieme (`String`: un dato con le sue operazioni).

Un modulo **esporta** metodi e costanti, cioè li mette a disposizione dei client, e può **importare** quelli di altri moduli. È composto da due parti, tenute separate:

- l'**interfaccia**: *cosa* fa il modulo e *come si usa*. È la sola parte visibile ai client. Di `Math.sqrt` è `double sqrt(double a)`
- il **corpo**: *come* l'astrazione è realizzata. Nascosto e protetto. Di `Math.sqrt` sono le istruzioni che calcolano la radice

Information hiding: al corpo si accede soltanto attraverso l'interfaccia, che espone lo stretto necessario e resta **stabile**. Finché non cambia, il corpo si può riscrivere senza toccare i client.



Java ha anche una parola chiave `interface`: un costrutto specifico (lezione 9).

Un modulo in Java, oggi – `Voti.java` e `ProvaVoti.java`

```java
/** Modulo: i voti d'esame in trentesimi. */
public class Voti {

    public static final int MINIMO = 18;
    public static final int MASSIMO = 30;

    /** true se v e' un voto valido. */
    public static boolean isValido(int v) {
        return v >= MINIMO && v <= MASSIMO;
    }

    /** La media riportata in 110-esimi. */
    public static int baseLaurea(double media) {
        return arrotonda(media * 110 / MASSIMO);
    }

    // corpo: visibile solo dentro Voti
    private static int arrotonda(double x) {
        return (int) (x + 0.5);
    }
}
```

Nel `main` di `ProvaVoti`:

```java
boolean ok = Voti.isValido(31);     // false
int base = Voti.baseLaurea(27.4);   // 100
System.out.println(ok + " " + base);
System.out.println(Voti.MASSIMO);   // 30
// Voti.arrotonda(2.5);  // ERRORE: private
```

```
$ javac ProvaVoti.java
$ java ProvaVoti
false 100
30
```

- un modulo: una **classe** di metodi `static` nel suo file, **senza** `main`
- `public` è l'interfaccia: di ogni metodo al client bastano la firma e il commento `/** */`
- `private` è il corpo: si usa solo dentro la classe
- il client, qui `ProvaVoti`, scrive `Voti.metodo(...)`
- i due file stanno nella stessa cartella: compilando `ProvaVoti.java`, `javac` compila anche `Voti.java`

Programmazione modulare: coesione e accoppiamento

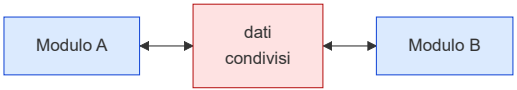
Coesione (dentro un modulo)

Un modulo è **fortemente coeso** se racchiude caratteristiche omogenee: realizza *una sola* astrazione. `Voti` sa tutto dei voti e nient'altro; `Math` solo di matematica.

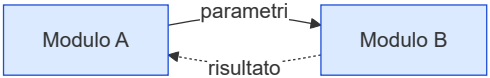
Accoppiamento (fra moduli)

Misura quanto un modulo dipende dagli altri. Due moduli sono **debolmente accoppiati** se ciascuno conosce dell'altro solo l'interfaccia: gli passa dei parametri e ne riceve un risultato. Sono fortemente accoppiati se lavorano sugli stessi dati: non si può cambiarne uno senza toccare l'altro.

accoppiamento **forte**



accoppiamento **debole**



L'obiettivo è **alta coesione e basso accoppiamento**, ma i due criteri tirano in direzioni opposte: moduli piccolissimi sono molto coesi, ma per fare qualcosa devono usarsi a vicenda di continuo. Le dipendenze si moltiplicano e l'accoppiamento cresce. Va cercato un compromesso.

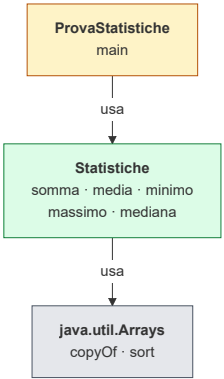
Commentiamo insieme (20 min) – il problema

Il problema. Molti programmi devono calcolare statistiche su sequenze di interi: voti, temperature, conteggi. Serve un modulo che lo faccia una volta per tutte.

Le regole

- il modulo calcola somma, media, minimo, massimo e mediana
- la *mediana* è il valore centrale dei dati ordinati; con un numero pari di valori, la media dei due centrali
- ogni calcolo si chiede con un array oppure direttamente con un elenco di valori: `media(voti)`, `media(18, 30)`
- i dati ricevuti non vengono mai modificati, nemmeno per ordinarli
- serve almeno un valore

La soluzione è già scritta: il modulo `Statistiche` e il programma `ProvaStatistiche`, che lo usa conoscendone soltanto l'interfaccia. Li leggiamo distinguendo interfaccia e corpo.



Commentiamo insieme – Statistiche.java (1/2)

```java
import java.util.Arrays;

/**
 * Modulo: statistiche su sequenze di interi.
 * Tutti i metodi LEGGONO gli argomenti.
 * Precondizione comune: almeno un valore.
 */
public class Statistiche {

    public static int somma(int... v) {
        int s = 0;
        for (int x : v) {
            s += x;
        }
        return s;
    }

    public static double media(int... v) {
        return (double) somma(v) / v.length;
    }
```

```java
    public static int minimo(int... v) {
        int min = v[0];
        for (int x : v) {
            if (x < min) {
                min = x;
            }
        }
        return min;
    }

    public static int massimo(int... v) {
        int max = v[0];
        for (int x : v) {
            if (x > max) {
                max = x;
            }
        }
        return max;
    }
```

Commentiamo insieme – Statistiche.java (2/2)

```
    /** Il valore centrale dei dati ordinati. */
    public static double mediana(int... v) {
        int[] o = ordinata(v);
        int meta = o.length / 2;
        if (o.length % 2 == 1) {
            return o[meta];   // int -> double
        }
        return (o[meta - 1] + o[meta]) / 2.0;
    }

    // corpo: CREA una copia ordinata di v
    private static int[] ordinata(int[] v) {
        int[] copia = Arrays.copyOf(v, v.length);
        Arrays.sort(copia);
        return copia;
    }
}
```

Da notare, leggendo:

- `int... v`: ogni metodo si chiama con un array oppure con un elenco di valori
- `media`: il cast è su `somma(v)`, prima della divisione
- `mediana`: `return o[meta]` restituisce un `int` dove è atteso un `double` (conversione implicita); con un numero pari di valori, la media dei due centrali: `/ 2.0` e non `/ 2`
- `ordinata` è `private`: appartiene al corpo. `Arrays.sort` *modifica*, il modulo promette di *leggere*: si ordina una copia
- l'interfaccia: cinque metodi `public` e il commento iniziale

Commentiamo insieme – ProvaStatistiche.java

```java
int[] voti = {28, 30, 24, 27, 30, 18};

System.out.println(Statistiche.somma(voti));
System.out.println(Statistiche.media(voti));
System.out.println(Statistiche.minimo(voti));
System.out.println(Statistiche.massimo(voti));
System.out.println(Statistiche.mediana(voti));
System.out.println(Arrays.toString(voti));

System.out.println(Statistiche.media(18, 30));
System.out.println(Statistiche.mediana(5, 1, 3));
int vicina = (int) (Statistiche.media(voti) + 0.5);
System.out.println(vicina);
```

```
157
26.166666666666668
18
30
27.5
[28, 30, 24, 27, 30, 18]
24.0
3.0
26
```

Rispondiamo insieme:

1. `Statistiche.media(18, 30)`: chi costruisce l'array? Che tipo ha `v` dentro `media`?
2. dopo `mediana(voti)` l'array `voti` è ancora nell'ordine originale: quale riga del modulo lo garantisce?
3. se in `media` togliamo `(double)`, cosa stampa la seconda riga?
4. possiamo scrivere qui `Statistiche.ordinata(voti)`? Perché?
5. `Statistiche.media()` compila? E `Statistiche.minimo()`? Cosa succede eseguendoli?

Insieme, passo passo (30 min) – il problema

Il problema. Leggere un testo, contare quante volte compare ogni lettera e segnalare la più frequente.

Le regole

- contano solo le lettere da a a z, senza distinguere maiuscole e minuscole; gli altri caratteri si ignorano
- per ogni lettera che compare almeno una volta si stampano, in ordine alfabetico, la lettera, quante volte compare e la sua percentuale sul totale delle lettere, con un decimale
- la lettera più frequente è segnata con `<-- max`; a pari merito, lo sono tutte

Si parte da un programma che risolve già il problema, ma con un `main` che fa tutto da solo. Lo riscriviamo insieme, a moduli.

Il risultato atteso:

```
$ java Frequenze
Testo: Java e la JVM
a   3  30.0%  <-- max
e   1  10.0%
j   2  20.0%
l   1  10.0%
m   1  10.0%
v   2  20.0%
```

10 lettere in tutto: la a compare 3 volte, cioè il 30%.

Insieme, passo passo – il punto di partenza: `FrequenzeMonolite.java`

```
Scanner in = new Scanner(System.in);
System.out.print("Testo: ");
String testo = in.nextLine().toLowerCase();
in.close();
String alfabeto = "abcdefghijklmnopqrstuvwxyz";
int[] conteggi = new int[26];
int totale = 0;
for (int i = 0; i < testo.length(); i++) {
    int p = alfabeto.indexOf(testo.charAt(i));
    if (p >= 0) {
        conteggi[p]++;
        totale++;
    }
}
```

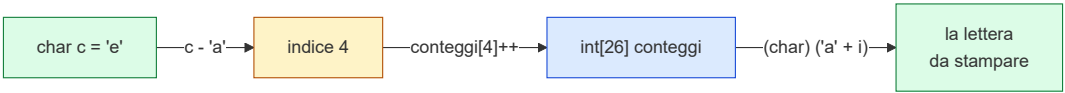
Questo `main` funziona, ed è scritto con i soli strumenti delle lezioni 1–3. Ma fa *tutto*: legge, conta le lettere, cerca il massimo, calcola le percentuali, stampa. Nessuna di queste parti si può riusare altrove, né provare da sola.

```
int max = conteggi[0];
for (int i = 1; i < 26; i++) {
    if (conteggi[i] > max) {
        max = conteggi[i];
    }
}
for (int i = 0; i < 26; i++) {
    int n = conteggi[i];
    if (n > 0) {
        double perc = 100.0 * n / totale;
        System.out.printf("%c %3d %5.1f%%",
            alfabeto.charAt(i), n, perc);
        if (n == max) {
            System.out.print("  <-- max");
        }
        System.out.println();
    }
}
```

Insieme, passo passo – dal `main` che fa tutto ai moduli

Prima di scrivere codice, decidiamo insieme:

1. quanti **compiti** diversi ha quel `main`? → ingresso e uscita; contare le lettere; sommare e trovare il massimo
2. quali **moduli**? → `Lettere` (nuovo): trasforma le lettere in indici e le conta; `Statistiche`: è già pronto, lo **riusiamo**; `Frequenze`: il `main`, solo ingresso e uscita
3. l'**interfaccia** di `Lettere`: cosa serve al client? → `int[] conta(String s)` e `char lettera(int indice)`; il resto è corpo, `private`
4. cosa cambia con gli strumenti di oggi? → `c - 'a'` al posto di `alfabeto.indexOf(c)`; `(char) ('a' + i)` al posto di `alfabeto.charAt(i)`; il cast al posto di `100.0 *`



Da tenere a mente: il programma nuovo deve stampare *esattamente* ciò che stampa il vecchio.

Una possibile soluzione (1/2) – il modulo `Lettere.java`

```java
/** Modulo: le 26 lettere come indici. */
public class Lettere {

    public static final int QUANTE = 26;

    /** L'indice 0 e' 'a', il 25 e' 'z'. */
    public static char lettera(int indice) {
        return (char) ('a' + indice);
    }

    /** CREA l'array dei 26 conteggi. */
    public static int[] conta(String s) {
        int[] conteggi = new int[QUANTE];
        for (int i = 0; i < s.length(); i++) {
            char c = s.charAt(i);
            c = Character.toLowerCase(c);
            if (isLettera(c)) {
                conteggi[indice(c)]++;
            }
        }
        return conteggi;
    }

    // corpo: solo le minuscole da 'a' a 'z'
    private static boolean isLettera(char c) {
        return c >= 'a' && c <= 'z';
    }

    // 'a' -> 0, 'b' -> 1, ..., 'z' -> 25
    private static int indice(char c) {
        return c - 'a';
    }
}
```

- interfaccia: `QUANTE`, `lettera`, `conta`. `isLettera` e `indice` al client non servono: `private`
- `c - 'a'` è già un `int`; `'a' + indice` anche, e per farne un `char` serve il cast
- perché non `Character.isLetter`? Per una e accentata risponde `true`, e l'indice sarebbe $232 - 97 = 135$: fuori dall'array
- niente `Scanner` né stampe: il modulo conta e basta

Una possibile soluzione (2/2) – il programma `Frequenze.java`

```java
public static void main(String[] args) {
    Scanner in = new Scanner(System.in);
    System.out.print("Testo: ");
    String testo = in.nextLine();
    in.close();

    int[] conteggi = Lettere.conta(testo);
    int totale = Statistiche.somma(conteggi);
    int max = Statistiche.massimo(conteggi);

    for (int i = 0; i < Lettere.QUANTE; i++) {
        int n = conteggi[i];
        if (n > 0) {
            double perc = (double) n / totale * 100;
            System.out.printf("%c %3d %5.1f%%",
                    Lettere.lettera(i), n, perc);
            if (n == max) {
                System.out.print("  <-- max");
            }
            System.out.println();
        }
    }
}
```

```
$ javac Frequenze.java
$ java Frequenze
Testo: Java e la JVM
a   3  30.0%  <-- max
e   1  10.0%
j   2  20.0%
l   1  10.0%
m   1  10.0%
v   2  20.0%
```

Tre righe al centro del `main` dicono *cosa* fa il programma; il *come* sta nei moduli.

`Statistiche` è nato per i voti e qui lavora sui conteggi delle lettere, senza una modifica: è il **riuso**.

Stesso output di `FrequenzeMonolite`: è cambiata la struttura, non il comportamento.

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	**Dove compare oggi**
Conversioni implicite, promozione (U11)	`Widening`; `return o[meta]` in `mediana`
Cast esplicito, troncamento (U11)	`Narrowing`; `media`; le percentuali in `Frequenze`
`char` e Unicode, aritmetica dei caratteri (U11)	`Caratteri`; `indice` e `lettera` in `Lettere`
Classi wrapper, costanti e metodi `static` (U11)	`Wrapper`
Autoboxing, unboxing, `==` ed `equals` (U11)	`Boxing`
La classe `Character` (U11)	`InfoCarattere`; `toLowerCase` in `conta`
`Arrays`: `toString`, `copyOf`, `equals`, `fill` (U11)	`UsaArrays`; `ordinata` in `Statistiche`
`Arrays`: `sort`, `binarySearch` (U11)	`OrdinaCerca`; `mediana`
Argomenti a lunghezza variabile (U11)	`Varargs`; tutti i metodi di `Statistiche`
Astrazione, modulo, interfaccia e corpo (U3)	`Voti` e `ProvaVoti`
Information hiding, `private` (U3)	`arrotonda`; `ordinata`; `isLettera` e `indice`
Coesione, accoppiamento, riuso (U3)	da `FrequenzeMonolite` a `Lettere` + `Statistiche` + `Frequenze`

Prossima lezione (L5): i concetti della programmazione a oggetti e le **classi** in Java: incapsulamento, costruttori, metodi `get` e `set`, ciclo di vita degli oggetti.

Homework 4 (da svolgere entro la prossima lezione)

1. `Anagrammi.java`: legge due frasi e dice se sono una l'anagramma dell'altra, ignorando spazi, punteggiatura e maiuscole (*"Calendario"* e *"locandiera"* lo sono). Un metodo `char[] lettereOrdinate(String s)` *crea* l'array delle sole lettere, rese minuscole con `Character` e ordinate con `Arrays.sort`; il confronto si fa con `Arrays.equals`.
2. `Pagella.java`: i voti arrivano dalla riga di comando (`java Pagella 28 30 24 27 30 18`). Un metodo `int[] inInteri(String[] testi)` li converte con `Integer.parseInt`; il programma stampa i voti, i voti ordinati (su una copia), la media con 2 decimali, la mediana, la media arrotondata all'intero e la base di laurea in 110-esimi. Usate il modulo `Statistiche` di oggi **senza modificarlo**: copiatelo nella cartella `homework/`.
3. `Cifrario.java` e `ProvaCifrario.java`: il cifrario di Cesare come modulo, senza la stringa `ALFABETO`: solo aritmetica dei `char` e cast. Interfaccia: `cifra(String, int)` e `decifra(String, int)`; nel corpo un metodo `private sposta(char, int)` che conserva maiuscole e minuscole. Il programma di prova legge testo e chiave, stampa il testo cifrato e quello decifrato, poi le 26 decifrature possibili del testo cifrato (*forza bruta*).

Per ogni metodo decidete se è interfaccia (`public`) o corpo (`private`). Usate solo ciò che abbiamo visto finora.

Domande?

mattia.fonisto@unina.it